

COMPLETE DATABASE SYSTEMS

Upper Secondary / Terminale / IT Coursebook

Databases • SQL • Design • Transactions • Security

30 Complete Lessons • Practice Activities
Revision Tests • Answer Key • Reference Tables

A structured 80-page learning book for classroom teaching, independent study and exam preparation.

COMPUTER SCIENCE SERIES

Course focus	Database concepts, data modeling, SQL, normalization, transactions, security, administration and modern database architectures.
Level	Upper Secondary / Terminale / IT & Computer Science.
Language	English.

GETTING STARTED

How to Use This Book

A practical roadmap for mastering Database Systems step by step.

What You Will Learn

- Database foundations, users, models and architecture.
- ER modeling, relational design, keys and normalization.
- SQL querying, joins, aggregation, subqueries and stored objects.
- Transactions, concurrency, indexing, security, backup and distributed systems.
- NoSQL, data warehousing, application connectivity, governance and project work.

Recommended Study Method

1. Read the lesson objectives before studying the concepts.
2. Recreate each example using a small school or business database.
3. Complete guided practice without looking at the answers.
4. Correct mistakes by writing the rule behind the correction.
5. Practice SQL and modeling repeatedly, not only by reading.

Database Editing Checklist

- Model: Does each table represent a clear subject?
- Keys: Is every row uniquely identifiable?
- Relationships: Are foreign keys correct?
- Integrity: Are NULL, uniqueness and domain rules appropriate?
- SQL: Are join conditions and filters correct?
- Transactions: Should multiple statements succeed or fail together?
- Security: Does each user have only needed privileges?
- Performance: Are important queries supported by suitable indexes?

Suggested 6-Week Study Schedule

- Week 1: Lessons 1-5 - foundations, roles, architecture, ER and relational model.
- Week 2: Lessons 6-10 - keys, algebra and core SQL.
- Week 3: Lessons 11-15 - grouping, subqueries, design and normalization.
- Week 4: Lessons 16-20 - advanced normalization, transactions, concurrency, indexes and views.
- Week 5: Lessons 21-25 - security, backup, storage, optimization and distributed databases.
- Week 6: Lessons 26-30 + revision tests and project review.

REFERENCE

Table of Contents

Lessons 1-15

Lesson	Topic	Pages
1	Introduction to Database Systems	5-6
2	Database Environment and Users	7-8
3	Data Models and Database Architecture	9-10
4	Entity-Relationship Modeling	11-12
5	Relational Model Fundamentals	13-14
6	Keys and Integrity Constraints	15-16
7	Relational Algebra	17-18
8	Introduction to SQL	19-20
9	SQL SELECT and Filtering	21-22
10	SQL Joins	23-24
11	SQL Aggregation and Grouping	25-26
12	SQL Subqueries and Set Operations	27-28
13	Database Design Process	29-30
14	Functional Dependencies	31-32
15	Normalization: First Normal Form	33-34

Part I Learning Map

- Lessons 1-5: Foundations, users, architecture, ER modeling and relational model.
- Lessons 6-10: Keys, relational algebra and core SQL.
- Lessons 11-15: Aggregation, subqueries, design and normalization.

Part I Milestones

- I can model entities, attributes and relationships.
- I can identify primary and foreign keys.
- I can write and interpret core SQL queries.
- I can explain why normalization reduces redundancy.

Part I Study Notes

REFERENCE

Table of Contents

Lessons 16-30, Revision and Reference

Lesson	Topic	Pages
16	Normalization: Second and Third Normal Forms	35-36
17	Boyce-Codd Normal Form and Denormalization	37-38
18	Transactions and ACID Properties	39-40
19	Concurrency Control	41-42
20	Indexes and Query Performance	43-44
21	Views and Stored Database Objects	45-46
22	Database Security and Authorization	47-48
23	Backup, Recovery and High Availability	49-50
24	Database Storage and File Organization	51-52
25	Query Processing and Optimization	53-54
26	Distributed Databases	55-56
27	NoSQL Databases	57-58
28	Data Warehousing and OLAP	59-60
29	Database APIs and Application Connectivity	61-62
30	Database Systems Review and Exam Preparation	63-64
Revision	Review Tests 1-8	65-72
Answers	Answer Key and Skills	73-78
Reference	Database Reference Tables	79
Reference	Database Quick Guide	80

Part II Learning Map

- Lessons 16-20: BCNF, transactions, concurrency, indexing and stored objects.
- Lessons 21-25: Security, recovery, storage, optimization and distributed databases.
- Lessons 26-30: NoSQL, warehousing, connectivity, governance and project review.

Part II Milestones

- I can explain ACID transactions and concurrency problems.
- I can justify indexes, security controls and backup strategies.
- I can compare relational and non-relational approaches.
- I can plan a small database project from requirements to testing.

Part II Study Notes

LESSON 01 - CORE DATABASE SYSTEMS

Introduction to Database Systems

Concepts, structures, rules and model examples

Learning Objectives

- Define a database and a database management system (DBMS).
- Explain why organizations use databases.
- Distinguish data, information and metadata.

Concept	Explanation	Example
Database	An organized collection of related data stored for efficient access and management.	A school database stores students, classes and grades.
DBMS	Software used to create, store, query, secure and maintain databases.	PostgreSQL, MySQL, SQL Server.
Data	Raw facts that can be recorded and processed.	Student ID, name, date of birth.
Information	Processed data that has meaning in context.	A report showing average grade by class.
Metadata	Data that describes other data.	Column names, data types and constraints.

Key Points

- A DBMS provides controlled access to shared data.
- Databases reduce unnecessary duplication and improve consistency.
- Database systems combine data, DBMS software, applications, users and procedures.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A library database can track books and loans.
2. A hospital database can store patients and appointments.
3. A DBMS can enforce rules such as unique student IDs.
4. Queries turn stored data into useful information.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 01 - PRACTICE

Introduction to Database Systems

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A spreadsheet is always the same as a database.	Spreadsheets and databases overlap in use, but a DBMS supports stronger structure, querying, concurrency and integrity.	Different tools serve different data-management needs.
Metadata means secret data.	Metadata describes the structure and properties of stored data.	It is information about data, not necessarily confidential data.

Guided Practice

1. What is a database?
2. What is a DBMS?
3. Give two advantages of using a DBMS.
4. Distinguish data from information.
5. What is metadata?
6. Name two examples of DBMS software.
7. Give one example of a database application.
8. Name three components of a database system.

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Introduction to Database Systems.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. An organized collection of related data.
2. Software that manages database creation, storage, queries, security and maintenance.
3. For example, reduced redundancy and improved consistency.
4. Data are raw facts; information is meaningful processed data.
5. Data describing other data.
6. For example, PostgreSQL and MySQL.
7. For example, a school records system.
8. For example, data, DBMS, users, applications and procedures.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 02 - CORE DATABASE SYSTEMS

Database Environment and Users

Concepts, structures, rules and model examples

Learning Objectives

- Identify major database users and roles.
- Explain the responsibilities of a database administrator (DBA).
- Describe the interaction between applications and the DBMS.

Concept	Explanation	Example
DBA	Person or team responsible for administration, security, backup, recovery and performance.	Creates users and manages privileges.
Database designer	Defines data structures, relationships and constraints.	Designs entities and tables.
Application developer	Builds programs that access the database.	Creates a student registration web app.
End user	Uses applications or queries to work with data.	Teacher viewing grades.
Database server	Computer system running database services.	A server hosting PostgreSQL.

Key Points

- Different roles separate design, operation, development and use.
- The DBA protects availability, security and recoverability.
- Applications communicate with the DBMS through drivers, APIs or database protocols.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A DBA schedules backups.
2. A designer models student-course relationships.
3. A developer writes parameterized SQL queries.
4. An end user may never see SQL directly.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 02 - PRACTICE

Database Environment and Users

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
The DBA only creates tables.	DBA duties also include security, backup, recovery, monitoring and performance.	Administration is broader than schema creation.
All users should receive administrator privileges.	Users should receive only the permissions they need.	Least privilege reduces security risk.

Guided Practice

1. Who is a DBA?
2. Give three DBA responsibilities.
3. What does a database designer do?
4. What does an application developer do?
5. Who is an end user?
6. Why is least privilege important?
7. What is a database server?
8. How can an application communicate with a DBMS?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Database Environment and Users.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A person or team responsible for database administration.
2. Security, backup/recovery and performance monitoring, among others.
3. Defines structures, relationships and constraints.
4. Builds software that accesses the database.
5. A person who uses the database through applications or queries.
6. It limits damage from mistakes or attacks.
7. A computer system running database services.
8. Through drivers, APIs or database protocols.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 03 - CORE DATABASE SYSTEMS

Data Models and Database Architecture

Concepts, structures, rules and model examples

Learning Objectives

- Describe common data models.
- Explain the three-schema architecture.
- Distinguish logical and physical data independence.

Concept	Explanation	Example
Data model	A set of concepts for describing data, relationships and constraints.	Relational model.
Relational model	Represents data as tables of rows and columns.	STUDENT(StudentID, Name, Class).
Conceptual schema	Global logical description of the database.	Entities, relationships and constraints.
External schema	User-specific view of the database.	Teacher view showing only assigned classes.
Internal schema	Physical storage description.	Files, pages and indexes.

Key Points

- Three-schema architecture separates user views, logical design and physical storage.
- Logical data independence allows changes to the logical schema with minimal effect on user views.
- Physical data independence allows storage changes without changing the logical schema.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A relational database stores students in tables.
2. A user view can hide sensitive columns.
3. Adding an index changes physical storage, not the logical table design.
4. Renaming a logical attribute may affect application views.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 03 - PRACTICE

Data Models and Database Architecture

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
The physical schema is the same as a user view.	The physical schema describes storage; user views are external schemas.	Different schema levels serve different purposes.
Data independence means data never changes.	Data independence means one schema level can change with limited impact on higher levels.	It concerns separation, not immutability.

Guided Practice

1. What is a data model?
2. Name three schema levels.
3. What does the external schema describe?
4. What does the internal schema describe?
5. Define physical data independence.
6. Define logical data independence.
7. Give one example of a relational structure.
8. Why is schema separation useful?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Data Models and Database Architecture.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A framework for describing data and relationships.
2. External, conceptual and internal.
3. User-specific views.
4. Physical storage details.
5. Changing storage without changing the logical schema.
6. Changing the logical schema with limited impact on user views.
7. A table such as STUDENT(StudentID, Name).
8. It reduces coupling between users, design and storage.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 04 - CORE DATABASE SYSTEMS

Entity-Relationship Modeling

Concepts, structures, rules and model examples

Learning Objectives

- Identify entities, attributes and relationships.
- Create simple ER models.
- Interpret cardinality and participation.

Concept	Explanation	Example
Entity	A distinguishable real-world object or concept.	Student, Course, Teacher.
Attribute	A property of an entity or relationship.	StudentName, BirthDate.
Relationship	An association between entities.	Student enrolls in Course.
Cardinality	Number of entity occurrences that may participate in a relationship.	One-to-many, many-to-many.
Key attribute	Attribute or set of attributes that uniquely identifies an entity.	StudentID.

Key Points

- ER modeling is used before table implementation to understand business data.
- Cardinality captures rules such as one department having many employees.
- Many-to-many relationships usually become associative tables in a relational design.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. STUDENT has attributes StudentID and Name.
2. COURSE is related to STUDENT through ENROLLMENT.
3. One teacher may teach many classes.
4. A course may have many students and a student may take many courses.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 04 - PRACTICE

Entity-Relationship Modeling

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
An attribute is always an entity.	Attributes describe entities; entities represent objects or concepts.	They play different modeling roles.
Many-to-many relationships can be implemented directly with one foreign key.	They normally require an associative table.	One foreign key cannot represent multiple values cleanly.

Guided Practice

1. Define entity.
2. Define attribute.
3. Define relationship.
4. What is cardinality?
5. Give an example of a one-to-many relationship.
6. Give an example of a many-to-many relationship.
7. What is a key attribute?
8. Why are ER models useful?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Entity-Relationship Modeling.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A distinguishable object or concept.
2. A property that describes an entity or relationship.
3. An association between entities.
4. The number of occurrences allowed in a relationship.
5. For example, Department to Employees.
6. For example, Students to Courses.
7. An attribute that uniquely identifies an entity.
8. They clarify data requirements before implementation.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 05 - CORE DATABASE SYSTEMS

Relational Model Fundamentals

Concepts, structures, rules and model examples

Learning Objectives

- Explain relations, tuples and attributes.
- Describe domains and relation schemas.
- Interpret relational tables correctly.

Concept	Explanation	Example
Relation	A set of tuples with the same attributes; implemented as a table.	STUDENT relation.
Tuple	One row in a relation.	One student record.
Attribute	Named column in a relation.	Name or BirthDate.
Domain	Set of valid values for an attribute.	Positive integers for Credits.
Relation schema	Name and attributes defining a relation.	COURSE(CourseID, Title, Credits).

Key Points

- In the formal relational model, tuples are unordered and duplicates are not allowed.
- Every attribute draws values from a defined domain.
- A table design should represent one coherent subject.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. One STUDENT row represents one student.
2. CourseID may use a text domain such as VARCHAR.
3. Credits may be restricted to values 1-6.
4. A relation schema defines structure, not the current rows.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 05 - PRACTICE

Relational Model Fundamentals

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Row order changes the meaning of a relation.	Relational meaning does not depend on row order.	Queries must use ORDER BY when ordering is required.
A domain is the same as a column name.	A domain is the set or type of allowed values; the attribute has a name.	Name and allowed values are different concepts.

Guided Practice

1. What is a relation?
2. What is a tuple?
3. What is an attribute?
4. What is a domain?
5. What is a relation schema?
6. Does row order matter in the relational model?
7. Why are domains important?
8. Give an example relation schema.

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Relational Model Fundamentals.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A set of tuples sharing the same attributes.
2. A row in a relation.
3. A named column.
4. The allowed set or type of values for an attribute.
5. The structural definition of a relation.
6. No, not formally.
7. They help enforce valid values.
8. For example, COURSE(CourseID, Title, Credits).

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 06 - CORE DATABASE SYSTEMS

Keys and Integrity Constraints

Concepts, structures, rules and model examples

Learning Objectives

- Distinguish superkeys, candidate keys and primary keys.
- Explain foreign keys.
- Describe entity and referential integrity.

Concept	Explanation	Example
Superkey	Any attribute set that uniquely identifies a row.	{StudentID} or {StudentID, Name}.
Candidate key	Minimal superkey.	StudentID.
Primary key	Candidate key chosen as the main identifier.	StudentID PRIMARY KEY.
Foreign key	Attribute referencing a key in another relation.	Enrollment.StudentID references Student.StudentID.
Referential integrity	Rule requiring valid references between related rows.	Enrollment cannot reference a missing student.

Key Points

- Primary key values must uniquely identify rows and should not be NULL.
- Foreign keys preserve relationships across tables.
- Constraints move important business rules into the database.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. CourseID identifies a course.
2. Enrollment may use a composite primary key (StudentID, CourseID).
3. DepartmentID in EMPLOYEE may reference DEPARTMENT.
4. Deleting a referenced parent row may be restricted or cascaded.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 06 - PRACTICE

Keys and Integrity Constraints

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A foreign key must always be unique.	Foreign key values may repeat unless another constraint requires uniqueness.	Many child rows can reference one parent.
A primary key can contain duplicate values.	Primary keys must be unique.	Uniqueness is the purpose of the key.

Guided Practice

1. What is a candidate key?
2. What is a primary key?
3. What is a foreign key?
4. What is referential integrity?
5. Can a foreign key repeat?
6. Why are primary keys important?
7. Give an example of a composite key.
8. What happens if referential integrity is violated?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Keys and Integrity Constraints.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A minimal set of attributes that uniquely identifies a row.
2. The candidate key selected as the main identifier.
3. An attribute that references a key in another relation.
4. The rule that references must point to valid related rows.
5. Yes, often.
6. They uniquely identify rows and support relationships.
7. For example, (StudentID, CourseID) in ENROLLMENT.
8. The DBMS should reject or otherwise handle the invalid operation according to constraints.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 07 - CORE DATABASE SYSTEMS

Relational Algebra

Concepts, structures, rules and model examples

Learning Objectives

- Describe basic relational algebra operations.
- Use selection and projection conceptually.
- Explain joins and set operations.

Concept	Explanation	Example
Selection	Chooses rows satisfying a condition.	Students where Class = "Terminale".
Projection	Chooses specified columns.	StudentID and Name only.
Join	Combines related tuples from two relations.	STUDENT joined to ENROLLMENT.
Union	Combines compatible rows from two relations.	CurrentStudents UNION Alumni.
Difference	Rows in one relation but not another.	RegisteredStudents minus PaidStudents.

Key Points

- Relational algebra provides a formal foundation for relational querying.
- Selection reduces rows; projection reduces columns.
- Join conditions determine how related tables are combined.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Select courses with Credits > 3.
2. Project only CourseID and Title.
3. Join ENROLLMENT with COURSE using CourseID.
4. Union requires compatible relation structures.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 07 - PRACTICE

Relational Algebra

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Projection filters rows.	Projection chooses columns; selection filters rows.	The operations have different roles.
A join always produces fewer rows.	A join can produce more, fewer or the same number depending on data and condition.	Result size depends on matching combinations.

Guided Practice

1. What does selection do?
2. What does projection do?
3. What does a join do?
4. When is union valid?
5. What does difference return?
6. Which operation reduces columns?
7. Which operation filters rows?
8. Why is relational algebra important?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Relational Algebra.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. It chooses rows satisfying a condition.
2. It chooses specified columns.
3. It combines related rows from relations.
4. When relations are union-compatible.
5. Rows in one relation but not the other.
6. Projection.
7. Selection.
8. It provides a formal foundation for relational query processing.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 08 - CORE DATABASE SYSTEMS

Introduction to SQL

Concepts, structures, rules and model examples

Learning Objectives

- Explain the purpose of SQL.
- Distinguish DDL, DML and DCL categories.
- Write simple CREATE, INSERT and SELECT statements conceptually.

Concept	Explanation	Example
SQL	Structured Query Language used to define, query and manage relational databases.	SELECT Name FROM Student;
DDL	Data Definition Language for schema objects.	CREATE TABLE, ALTER TABLE.
DML	Data Manipulation Language for rows.	SELECT, INSERT, UPDATE, DELETE.
DCL	Data Control Language for permissions.	GRANT, REVOKE.
TCL	Transaction Control Language.	COMMIT, ROLLBACK.

Key Points

- SQL is declarative: users state what result they need more than how to retrieve it.
- SQL syntax differs slightly across DBMS products.
- Keywords are often written uppercase for readability but usually are not case-sensitive.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. CREATE TABLE defines a table.
2. INSERT adds rows.
3. SELECT retrieves data.
4. COMMIT makes a transaction permanent.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 08 - PRACTICE

Introduction to SQL

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
SELECT changes table structure.	SELECT retrieves data; DDL statements such as ALTER change structure.	SQL categories serve different purposes.
SQL means every DBMS behaves identically.	DBMS products share core SQL but also have extensions and differences.	Portability is high but not perfect.

Guided Practice

1. What is SQL?
2. What is DDL?
3. Give two DDL statements.
4. What is DML?
5. Give two DML statements.
6. What does DCL control?
7. What does COMMIT do?
8. Why is SQL called declarative?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Introduction to SQL.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Structured Query Language for relational databases.
2. Language for defining database structures.
3. CREATE and ALTER.
4. Language for manipulating and querying data.
5. SELECT and INSERT, for example.
6. Permissions and access rights.
7. Makes transaction changes permanent.
8. Because users specify desired results rather than step-by-step retrieval procedures.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 09 - CORE DATABASE SYSTEMS

SQL SELECT and Filtering

Concepts, structures, rules and model examples

Learning Objectives

- Retrieve rows with SELECT.
- Filter data with WHERE.
- Sort and limit results.

Concept	Explanation	Example
SELECT	Chooses columns or expressions to return.	SELECT Name, Class FROM Student;
WHERE	Filters rows according to a condition.	WHERE Class = "Terminale".
ORDER BY	Sorts query results.	ORDER BY Name ASC.
DISTINCT	Removes duplicate result rows.	SELECT DISTINCT Class FROM Student;
Predicate	Condition that evaluates true, false or unknown.	Age >= 18 AND Active = TRUE.

Key Points

- Without ORDER BY, result order should not be assumed.
- WHERE is applied to individual rows before aggregation.
- NULL requires IS NULL or IS NOT NULL rather than ordinary equality.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. SELECT * returns all columns.
2. WHERE Price BETWEEN 10 AND 20 filters a range.
3. LIKE can perform pattern matching.
4. ORDER BY Grade DESC sorts highest first.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 09 - PRACTICE

SQL SELECT and Filtering

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
WHERE Name = NULL finds missing names.	Use WHERE Name IS NULL.	NULL represents unknown or missing and uses special logic.
ORDER BY permanently changes table row order.	ORDER BY sorts only the query result.	Physical row order is not guaranteed.

Guided Practice

1. What does SELECT do?
2. What does WHERE do?
3. What does DISTINCT do?
4. How do you sort results?
5. How do you test for NULL?
6. What does LIKE do?
7. What does DESC mean?
8. Is result order guaranteed without ORDER BY?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from SQL SELECT and Filtering.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Returns selected columns or expressions.
2. Filters rows.
3. Removes duplicate result rows.
4. Use ORDER BY.
5. Use IS NULL or IS NOT NULL.
6. Performs pattern matching.
7. Descending order.
8. No.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 10 - CORE DATABASE SYSTEMS

SQL Joins

Concepts, structures, rules and model examples

Learning Objectives

- Explain why joins are needed.
- Distinguish inner and outer joins.
- Write correct conceptual join conditions.

Concept	Explanation	Example
INNER JOIN	Returns matching rows from both sides.	Students with matching enrollments.
LEFT JOIN	Returns all left rows plus matching right rows.	All students, even those with no enrollment.
RIGHT JOIN	Returns all right rows plus matching left rows.	Supported by many DBMSs.
FULL OUTER JOIN	Returns matched and unmatched rows from both sides.	Complete comparison of two datasets.
Join condition	Expression defining how rows correspond.	Student.StudentID = Enrollment.StudentID.

Key Points

- Joins reconstruct related information stored across normalized tables.
- Missing join conditions can create a Cartesian product.
- Outer joins preserve unmatched rows from one or both inputs.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Join STUDENT to ENROLLMENT using StudentID.
2. LEFT JOIN can find students without courses.
3. INNER JOIN returns only matched rows.
4. Aliases make long queries easier to read.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 10 - PRACTICE

SQL Joins

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A join condition is optional and harmless to omit.	Omitting a needed condition can produce a large Cartesian product.	Rows combine with every row on the other side.
LEFT JOIN removes unmatched left rows.	LEFT JOIN preserves all left rows.	That is its defining behavior.

Guided Practice

1. What does INNER JOIN return?
2. What does LEFT JOIN preserve?
3. What is a join condition?
4. What is a Cartesian product?
5. Why are aliases useful?
6. Which join can keep unmatched rows from both sides?
7. How can you find students with no enrollment?
8. Why are joins common in normalized databases?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from SQL Joins.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Rows with matches on both sides.
2. All rows from the left table.
3. The expression defining matching rows.
4. Every row from one input combined with every row from the other.
5. They improve readability and disambiguation.
6. FULL OUTER JOIN.
7. Use LEFT JOIN and test the right-side key for NULL.
8. Related facts are stored in separate tables.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 11 - CORE DATABASE SYSTEMS

SQL Aggregation and Grouping

Concepts, structures, rules and model examples

Learning Objectives

- Use aggregate functions.
- Group rows with GROUP BY.
- Filter groups with HAVING.

Concept	Explanation	Example
COUNT	Counts rows or non-NULL values depending on expression.	COUNT(*) counts rows.
SUM	Adds numeric values.	SUM(Amount).
AVG	Calculates average of non-NULL numeric values.	AVG(Grade).
GROUP BY	Forms groups for aggregate calculation.	GROUP BY Class.
HAVING	Filters groups after aggregation.	HAVING COUNT(*) > 10.

Key Points

- WHERE filters rows before grouping; HAVING filters groups after aggregation.
- Aggregate functions usually ignore NULL values except COUNT(*).
- Every selected non-aggregate column normally must appear in GROUP BY.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Count students per class.
2. Average grade per subject.
3. Find classes with more than 30 students.
4. SUM can calculate total payments.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 11 - PRACTICE

SQL Aggregation and Grouping

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
HAVING and WHERE are identical.	WHERE filters rows; HAVING filters aggregated groups.	They operate at different stages.
COUNT(column) counts NULLs.	COUNT(column) normally ignores NULL; COUNT(*) counts rows.	The expression matters.

Guided Practice

1. What does COUNT(*) return?
2. What does AVG do?
3. What is GROUP BY used for?
4. What is HAVING used for?
5. Which clause filters rows before grouping?
6. Does COUNT(column) count NULL values?
7. What rule applies to non-aggregate selected columns?
8. Give one example of an aggregate query.

Mastery Challenge

- Write two original explanations demonstrating the main ideas from SQL Aggregation and Grouping.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Number of rows.
2. Average of non-NULL numeric values.
3. To form groups for aggregate calculation.
4. To filter groups after aggregation.
5. WHERE.
6. No, normally not.
7. They generally must appear in GROUP BY.
8. For example, average grade per class.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

Concepts, structures, rules and model examples

- Use scalar and multi-row subqueries conceptually.
- Explain IN, EXISTS and correlated subqueries.
- Use UNION, INTERSECT and EXCEPT conceptually.

Key Points

- Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

1. Find students above the average grade using a scalar subquery.
2. Use EXISTS to find courses that have enrollments.
3. Use UNION to combine similar lists.
4. Use EXCEPT to find rows present in one result but not another.

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Write one accurate database example and underline the key technical term.

LESSON 12 - PRACTICE

SQL Subqueries and Set Operations

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A scalar subquery can return many columns and rows.	A scalar subquery must yield one value.	It is used where a single value is expected.
UNION requires identical table names.	UNION requires compatible result columns, not identical source tables.	Compatibility is about shape and data types.

Guided Practice

1. What is a subquery?
2. What is a scalar subquery?
3. What does EXISTS test?
4. What is a correlated subquery?
5. What does UNION do?
6. What must UNION-compatible queries share?
7. What does EXCEPT return?
8. Give one use of a subquery.

Mastery Challenge

- Write two original explanations demonstrating the main ideas from SQL Subqueries and Set Operations.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A query nested inside another statement.
2. A subquery returning one value.
3. Whether the subquery returns at least one row.
4. A subquery that references the outer query row.
5. Combines compatible result sets, normally removing duplicates.
6. Same number of compatible columns.
7. Rows in the first result but not the second.
8. For example, comparing a value to an average computed by another query.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 13 - CORE DATABASE SYSTEMS

Database Design Process

Concepts, structures, rules and model examples

Learning Objectives

- Describe the stages of database design.
- Translate requirements into a conceptual model.
- Map conceptual models to relations.

Concept	Explanation	Example
Requirements analysis	Collects data needs, business rules and user tasks.	Interview users and review forms.
Conceptual design	Creates a technology-independent model.	ER diagram.
Logical design	Maps the conceptual model to a database model.	Convert entities to relational tables.
Physical design	Chooses storage structures, indexes and implementation details.	Index StudentID.
Validation	Checks design against rules and user needs.	Test sample transactions and queries.

Key Points

- Good design starts with requirements, not SQL coding.
- Business rules should be documented explicitly.
- Design is iterative; feedback may require changes.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A school identifies Student, Course and Enrollment entities.
2. Many-to-many relationships become associative tables.
3. Primary and foreign keys are chosen during logical design.
4. Indexes are considered during physical design.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 13 - PRACTICE

Database Design Process

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Start by creating tables immediately.	Start by understanding requirements and business rules.	Premature implementation can produce poor structure.
Physical design comes before conceptual design.	Conceptual and logical design normally precede physical design.	Storage choices depend on logical needs.

Guided Practice

1. What is requirements analysis?
2. What is conceptual design?
3. What is logical design?
4. What is physical design?
5. Why document business rules?
6. When are indexes typically considered?
7. What does validation check?
8. Why is database design iterative?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Database Design Process.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Collecting data needs, rules and tasks.
2. Creating a high-level model such as an ER model.
3. Mapping the conceptual model to database structures.
4. Choosing implementation and storage details.
5. So the database can enforce and reflect real requirements.
6. During physical design.
7. Whether the design supports rules and user needs.
8. New information and testing may require refinement.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 14 - CORE DATABASE SYSTEMS

Functional Dependencies

Concepts, structures, rules and model examples

Learning Objectives

- Define functional dependency.
- Identify determinants.
- Relate dependencies to keys and normalization.

Concept	Explanation	Example
Functional dependency	Relationship where one attribute set determines another.	StudentID → StudentName.
Determinant	Attribute set on the left side of a functional dependency.	StudentID in StudentID → Name.
Full dependency	Attribute depends on the whole composite key.	(StudentID, CourseID) → Grade.
Partial dependency	Non-key attribute depends on part of a composite key.	StudentID → StudentName in an enrollment table.
Transitive dependency	Non-key attribute depends on another non-key attribute.	StudentID → DeptID and DeptID → DeptName.

Key Points

- Functional dependencies describe data semantics, not just sample coincidences.
- Candidate keys functionally determine all attributes in a relation.
- Partial and transitive dependencies can indicate normalization problems.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. StudentID determines StudentName.
2. CourseID determines CourseTitle.
3. A composite enrollment key may determine Grade.
4. DepartmentID determines DepartmentName.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 14 - PRACTICE

Functional Dependencies

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
If two sample rows match once, there is automatically a functional dependency.	A functional dependency must follow the data meaning for all valid states.	Sample data alone is not enough.
Every dependency is a problem.	Dependencies are normal; certain dependency patterns cause redundancy.	Normalization uses them to improve design.

Guided Practice

1. Define functional dependency.
2. What is a determinant?
3. Give an example of a dependency.
4. What is a partial dependency?
5. What is a transitive dependency?
6. How are candidate keys related to dependencies?
7. Can sample data alone prove a functional dependency?
8. Why are dependencies important?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Functional Dependencies.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A relationship where one attribute set determines another.
2. The attribute set on the left side.
3. For example, StudentID -> StudentName.
4. A dependency on only part of a composite key.
5. A dependency through another non-key attribute.
6. A candidate key determines all attributes in its relation.
7. No.
8. They guide normalization and reveal redundancy.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 15 - CORE DATABASE SYSTEMS

Normalization: First Normal Form

Concepts, structures, rules and model examples

Learning Objectives

- Explain why normalization is used.
- Define first normal form (1NF).
- Remove repeating groups and multi-valued fields.

Concept	Explanation	Example
Normalization	Process of structuring relations to reduce redundancy and anomalies.	Decompose poorly designed tables.
1NF	Relation where attributes contain atomic values and there are no repeating groups.	One phone number per row/value field.
Repeating group	Multiple similar values stored in one row structure.	Phone1, Phone2, Phone3.
Atomic value	A value treated as a single indivisible value for the database design.	One CourseID per enrollment row.
Update anomaly	Problem where redundant data must be changed in several places.	Department name repeated in many rows.

Key Points

- 1NF is the starting point for formal normalization in relational design.
- Repeating groups are normally moved to separate related rows or tables.
- Normalization reduces anomalies but design choices still depend on workload and requirements.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Store multiple student phone numbers in a separate StudentPhone table.
2. Create one enrollment row per student-course pair.
3. Avoid comma-separated lists for values that must be queried individually.
4. Use primary keys to identify normalized rows.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 15 - PRACTICE

Normalization: First Normal Form

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
1NF means every table has one row.	1NF concerns atomic values and no repeating groups.	Row count is irrelevant.
Comma-separated CourseIDs are good relational design.	Store each course relationship as a separate related row.	Atomic, queryable values are preferred.

Guided Practice

1. Why normalize data?
2. What is 1NF?
3. What is a repeating group?
4. What is an atomic value?
5. Give one example of a 1NF violation.
6. What is an update anomaly?
7. How can multiple phone numbers be modeled?
8. Does normalization eliminate every performance problem?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Normalization: First Normal Form.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. To reduce redundancy and update anomalies.
2. Atomic values with no repeating groups.
3. Repeated similar attributes or multiple values in one row structure.
4. A value treated as one indivisible database value.
5. For example, storing several CourseIDs in one field.
6. A problem caused by needing to update redundant copies consistently.
7. Use a related StudentPhone table.
8. No.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 16 - CORE DATABASE SYSTEMS

Normalization: Second and Third Normal Forms

Concepts, structures, rules and model examples

Learning Objectives

- Define 2NF and 3NF.
- Identify partial and transitive dependencies.
- Decompose relations while preserving meaning.

Concept	Explanation	Example
2NF	1NF plus no partial dependency of non-key attributes on part of a candidate key.	Separate StudentName from Enrollment(StudentID, CourseID, Grade).
3NF	2NF plus no inappropriate transitive dependency of non-key attributes on a key.	Move DeptName to DEPARTMENT.
Decomposition	Splitting a relation into smaller relations.	STUDENT and DEPARTMENT.
Lossless join	Decomposition that allows original information to be reconstructed without spurious rows.	Join decomposed tables on a key.
Dependency preservation	Important dependencies can still be enforced after decomposition.	Constraints remain manageable.

Key Points

- 2NF matters mainly when candidate keys are composite.
- 3NF targets transitive dependencies among non-key attributes.
- Good decomposition aims to be lossless and preferably dependency-preserving.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Move CourseTitle to COURSE instead of repeating it in ENROLLMENT.
2. Move DepartmentName to DEPARTMENT when DeptID determines it.
3. Keep Grade in ENROLLMENT because it depends on the student-course pair.
4. Use foreign keys to connect decomposed relations.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 16 - PRACTICE

Normalization: Second and Third Normal Forms

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Every 1NF table is automatically 3NF.	1NF does not remove partial or transitive dependencies.	Higher normal forms add stricter requirements.
Normalization means splitting tables randomly.	Decomposition should follow dependencies and preserve information.	Random splits can lose meaning or create spurious joins.

Guided Practice

1. Define 2NF.
2. Define 3NF.
3. What is a partial dependency?
4. What is a transitive dependency?
5. What is lossless decomposition?
6. Why use foreign keys after decomposition?
7. Give one example of a 3NF decomposition.
8. Why is 2NF especially relevant to composite keys?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Normalization: Second and Third Normal Forms.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. 1NF with no partial dependency of non-key attributes on part of a candidate key.
2. 2NF with no inappropriate transitive dependency of non-key attributes on keys.
3. A dependency on part of a composite key.
4. A dependency through another non-key attribute.
5. A split that can reconstruct the original valid information without spurious rows.
6. To preserve relationships between new tables.
7. For example, STUDENT(StudentID, DeptID) and DEPARTMENT(DeptID, DeptName).
8. Because partial dependency requires a key with multiple attributes.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 17 - CORE DATABASE SYSTEMS

Boyce-Codd Normal Form and Denormalization

Concepts, structures, rules and model examples

Learning Objectives

- Explain BCNF at an introductory level.
- Recognize when 3NF may still contain anomalies.
- Explain controlled denormalization.

Concept	Explanation	Example
BCNF	A relation is in BCNF if every non-trivial functional dependency has a determinant that is a superkey.	Stricter than 3NF in some cases.
Anomaly	Unexpected redundancy or inconsistency risk caused by design.	Repeated assignment facts.
Denormalization	Intentional introduction of redundancy for performance or simplicity.	Precomputed summary table.
Materialized summary	Stored result derived from base data.	Daily sales totals.
Trade-off	Balance between consistency, storage, complexity and speed.	More joins versus more redundancy.

Key Points

- BCNF addresses certain dependency patterns not removed by 3NF.
- Denormalization should be deliberate and controlled, not accidental.
- Integrity mechanisms are essential when redundant copies are stored.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A reporting warehouse may store summaries for fast queries.
2. Triggers or batch jobs can maintain redundant derived data.
3. Highly normalized OLTP systems often prioritize consistency.
4. Performance testing should guide denormalization decisions.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 17 - PRACTICE

Boyce-Codd Normal Form and Denormalization

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
BCNF is always required for every practical table.	BCNF is a useful design target, but practical systems may use other forms based on constraints and needs.	Design involves trade-offs.
Denormalization means poor design.	Denormalization can be intentional when benefits outweigh costs.	Controlled redundancy can improve performance.

Guided Practice

1. What is BCNF?
2. How is BCNF related to 3NF?
3. What is denormalization?
4. Why might a system denormalize data?
5. What risk does redundancy introduce?
6. What is a materialized summary?
7. What should guide denormalization?
8. Why are integrity mechanisms important?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Boyce-Codd Normal Form and Denormalization.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Every determinant of a non-trivial dependency must be a superkey.
2. BCNF is stricter in some dependency cases.
3. Intentional introduction of redundancy.
4. To improve read performance or simplify reporting.
5. Inconsistency and update anomalies.
6. A stored result derived from base data.
7. Measured workload and performance needs.
8. They keep redundant copies consistent.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 18 - CORE DATABASE SYSTEMS

Transactions and ACID Properties

Concepts, structures, rules and model examples

Learning Objectives

- Define a transaction.
- Explain ACID properties.
- Use COMMIT and ROLLBACK conceptually.

Concept	Explanation	Example
Transaction	A logical unit of work that should be treated as one operation.	Transfer money between accounts.
Atomicity	All transaction operations succeed or none are applied.	Both debit and credit occur together.
Consistency	Transaction preserves defined database rules.	Balances remain valid according to constraints.
Isolation	Concurrent transactions should not improperly interfere.	Intermediate changes are controlled.
Durability	Committed changes survive failures.	Committed order remains stored after restart.

Key Points

- Transactions protect multi-step operations.
- COMMIT makes successful changes durable; ROLLBACK undoes uncommitted changes.
- ACID properties are implemented through logging, locking, MVCC and recovery mechanisms.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A bank transfer is one transaction.
2. An order and its items may be inserted together.
3. ROLLBACK can cancel a failed multi-step update.
4. COMMIT marks a successful transaction boundary.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 18 - PRACTICE

Transactions and ACID Properties

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A transaction is just one SQL statement.	A transaction may contain one or many statements.	It represents a logical unit of work.
COMMIT can be used to undo changes.	ROLLBACK undoes uncommitted work; COMMIT makes it permanent.	They have opposite purposes.

Guided Practice

1. What is a transaction?
2. What does atomicity mean?
3. What does consistency mean?
4. What does isolation mean?
5. What does durability mean?
6. What does COMMIT do?
7. What does ROLLBACK do?
8. Give one real-world transaction example.

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Transactions and ACID Properties.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A logical unit of database work.
2. All operations succeed together or none are applied.
3. Defined rules remain valid.
4. Concurrent transactions do not improperly interfere.
5. Committed changes survive failures.
6. Makes successful changes permanent.
7. Undoes uncommitted changes.
8. For example, a bank transfer.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 19 - CORE DATABASE SYSTEMS

Concurrency Control

Concepts, structures, rules and model examples

Learning Objectives

- Explain why concurrency control is needed.
- Identify common concurrency anomalies.
- Describe locking and MVCC at a basic level.

Concept	Explanation	Example
Lost update	One transaction overwrites another transaction's update.	Two users edit the same balance.
Dirty read	A transaction reads data another transaction has not committed.	Reading a temporary value that may roll back.
Non-repeatable read	Re-reading a row gives a different committed value.	Another transaction updated it.
Lock	Mechanism restricting simultaneous conflicting operations.	Exclusive write lock.
MVCC	Multi-version concurrency control using row versions to support isolation.	Readers can see a consistent snapshot.

Key Points

- Concurrency improves throughput but requires control to protect correctness.
- Isolation levels trade consistency guarantees against concurrency and overhead.
- Deadlocks can occur when transactions wait for each other in a cycle.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Row locking prevents conflicting writes.
2. MVCC can reduce reader-writer blocking.
3. A deadlock detector may abort one transaction.
4. Short transactions reduce lock contention.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 19 - PRACTICE

Concurrency Control

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Concurrency means transactions must run one at a time.	DBMSs allow concurrency while controlling conflicts.	Serial execution would limit performance.
A deadlock is the same as slow performance.	A deadlock is a cycle of transactions waiting for each other.	It is a specific concurrency condition.

Guided Practice

1. What is a lost update?
2. What is a dirty read?
3. What is a non-repeatable read?
4. What is a lock?
5. What is MVCC?
6. What is a deadlock?
7. Why use isolation levels?
8. Give one way to reduce lock contention.

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Concurrency Control.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. An update is overwritten by another concurrent update.
2. Reading uncommitted data.
3. Reading the same row twice and seeing different committed values.
4. A mechanism controlling conflicting access.
5. A concurrency approach based on multiple row versions.
6. A cycle of transactions waiting on each other.
7. To choose appropriate consistency guarantees and concurrency.
8. Keep transactions short, among other techniques.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 20 - CORE DATABASE SYSTEMS

Indexes and Query Performance

Concepts, structures, rules and model examples

Learning Objectives

- Explain the purpose of indexes.
- Compare indexed lookup with full table scanning.
- Identify index trade-offs.

Concept	Explanation	Example
Index	Auxiliary data structure that speeds retrieval for selected columns.	B-tree index on StudentID.
B-tree	Balanced tree structure commonly used for equality and range searches.	Index on Date or Name.
Full table scan	Reads many or all table rows to find matches.	May be efficient for small tables.
Selectivity	How strongly a condition narrows rows.	StudentID is usually highly selective.
Query plan	Strategy chosen by the optimizer to execute a query.	Index seek plus join.

Key Points

- Indexes speed many reads but require storage and maintenance during writes.
- An index is most useful when query patterns match its leading columns and predicates.
- The optimizer estimates costs and selects a query plan.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Primary keys are often indexed automatically.
2. An index on LastName may help name lookups.
3. Too many indexes can slow INSERT/UPDATE/DELETE.
4. A small table may be faster to scan than to use an index.

Database Analysis Routine

- Step 1: Identify the data, rule, query or system requirement.
- Step 2: Name the relevant database concept or structure.
- Step 3: Explain how the concept protects correctness, performance or usability.
- Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 20 - PRACTICE

Indexes and Query Performance

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Every column should be indexed.	Indexes should match important query and constraint needs.	Each index has write and storage costs.
An index guarantees the optimizer will use it.	The optimizer may choose a scan if it estimates that to be cheaper.	Plan choice depends on data and query conditions.

Guided Practice

1. What is an index?
2. What is a B-tree?
3. What is a full table scan?
4. What is selectivity?
5. Name one index benefit.
6. Name one index cost.
7. What is a query plan?
8. Why might the optimizer ignore an index?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Indexes and Query Performance.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A structure that speeds selected data access.
2. A balanced tree index structure.
3. Reading many or all rows directly.
4. How strongly a condition reduces matching rows.
5. Faster retrieval for matching queries.
6. Extra storage and write maintenance.
7. The execution strategy for a query.
8. A scan may be estimated cheaper.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 21 - CORE DATABASE SYSTEMS

Views and Stored Database Objects

Concepts, structures, rules and model examples

Learning Objectives

- Explain database views.
- Describe stored procedures, functions and triggers conceptually.
- Identify appropriate uses and risks.

Concept	Explanation	Example
View	Stored query presented like a virtual table.	ActiveStudents view.
Stored procedure	Named database program that performs operations.	RegisterStudent procedure.
Function	Routine that returns a value or result.	CalculateAge function.
Trigger	Automatic action fired by a database event.	Audit update after row change.
Materialized view	Stored query result refreshed periodically or on demand.	Precomputed sales summary.

Key Points

- Views can simplify access and hide columns or complex joins.
- Triggers are powerful but can make behavior less visible, so they should be documented carefully.
- Stored routines can centralize business logic close to data.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A view may show only non-sensitive employee fields.
2. A trigger can log changes to salary.
3. A stored procedure can perform a multi-step registration.
4. A materialized view can speed heavy reporting queries.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 21 - PRACTICE

Views and Stored Database Objects

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A view always stores a separate copy of data.	A normal view stores a query definition; a materialized view stores results.	They differ in storage behavior.
Triggers should contain every business rule.	Triggers are one tool; rules may belong in constraints, application logic or procedures.	Clarity and maintainability matter.

Guided Practice

1. What is a view?
2. What is a stored procedure?
3. What is a function?
4. What is a trigger?
5. What is a materialized view?
6. Give one security use of a view.
7. Give one use of a trigger.
8. Why should triggers be documented?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Views and Stored Database Objects.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A stored query presented like a virtual table.
2. A named database program for operations.
3. A routine that returns a value or result.
4. An automatic action fired by a database event.
5. A stored query result refreshed according to policy.
6. Hide sensitive columns or rows.
7. Audit changes, for example.
8. Because automatic hidden behavior can be difficult to understand and troubleshoot.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 22 - CORE DATABASE SYSTEMS

Database Security and Authorization

Concepts, structures, rules and model examples

Learning Objectives

- Explain authentication and authorization.
- Use roles and privileges conceptually.
- Apply least privilege.

Concept	Explanation	Example
Authentication	Verifies identity.	Password, certificate or federated login.
Authorization	Determines what an authenticated identity may do.	SELECT allowed; DELETE denied.
Role	Named collection of privileges assigned to users.	TeacherRole.
Privilege	Permission to perform an action on an object.	SELECT on Grades.
Least privilege	Grant only the access needed for a task.	Cashier cannot alter user accounts.

Key Points

- Strong security combines identity controls, privileges, auditing and secure configuration.
- Roles simplify privilege management.
- Sensitive data may need encryption at rest and in transit.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. GRANT can assign permissions.
2. REVOKE can remove permissions.
3. Audit logs record important security events.
4. Applications should avoid using administrator credentials.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 22 - PRACTICE

Database Security and Authorization

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Authentication and authorization mean the same thing.	Authentication checks identity; authorization checks allowed actions.	They are separate security stages.
Application accounts should always be DB administrators.	Application accounts should have limited role-based privileges.	Least privilege reduces impact of compromise.

Guided Practice

- | | |
|----------------------------|---|
| 1. What is authentication? | 5. Define least privilege. |
| 2. What is authorization? | 6. Why use roles? |
| 3. What is a role? | 7. What does GRANT do? |
| 4. What is a privilege? | 8. Why avoid administrator credentials in applications? |

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Database Security and Authorization.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Verification of identity.
2. Determination of allowed actions.
3. A named collection of privileges.
4. Permission to perform a database action.
5. Grant only needed access.
6. They simplify consistent permission management.
7. Assigns privileges.
8. Compromise would have excessive impact.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 23 - CORE DATABASE SYSTEMS

Backup, Recovery and High Availability

Concepts, structures, rules and model examples

Learning Objectives

- Explain backup and recovery goals.
- Distinguish full and incremental backup concepts.
- Describe high availability at a basic level.

Concept	Explanation	Example
Backup	Copy of data used for restoration after loss or corruption.	Nightly database backup.
Full backup	Backup of all selected database data.	Weekly full backup.
Incremental backup	Backup of changes since a previous backup point, depending on tool.	Daily changed blocks.
Recovery	Process of restoring data and bringing the database to a usable state.	Restore backup and replay logs.
High availability	Design that reduces service downtime.	Replication and failover.

Key Points

- Backups must be tested by performing restores.
- Recovery objectives should define acceptable data loss and downtime.
- Replication improves availability but is not a substitute for independent backups.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Store backup copies separately from the production server.
2. Use transaction logs for point-in-time recovery when supported.
3. Test disaster-recovery procedures.
4. Monitor replicas and failover health.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 23 - PRACTICE

Backup, Recovery and High Availability

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A backup is useful even if never tested.	Backups should be verified through restore tests.	An unreadable or incomplete backup may fail during disaster.
Replication replaces backups.	Replication can copy mistakes or corruption; backups provide historical recovery points.	The controls solve different problems.

Guided Practice

1. What is a backup?
2. What is recovery?
3. What is a full backup?
4. What is an incremental backup?
5. Why test restores?
6. What is high availability?
7. Why is replication not a backup?
8. What should recovery objectives define?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Backup, Recovery and High Availability.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A copy of data for restoration.
2. Restoring data and database service after failure.
3. A backup of all selected data.
4. A backup of changes according to the backup method.
5. To verify backups are usable.
6. Design for reduced downtime.
7. It may replicate deletion, corruption or errors.
8. Acceptable data loss and downtime.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 24 - CORE DATABASE SYSTEMS

Database Storage and File Organization

Concepts, structures, rules and model examples

Learning Objectives

- Describe pages, records and files at a basic level.
- Explain heap and ordered storage concepts.
- Relate storage layout to performance.

Concept	Explanation	Example
Page	Fixed-size block used by many DBMS storage engines.	8 KB page in some systems.
Record	Stored representation of a row.	One employee row.
Heap organization	Rows stored without a logical ordering key.	New rows placed where space is available.
Clustered organization	Rows or pages arranged according to a key or index structure, depending on DBMS.	Orders near OrderDate order.
Buffer cache	Memory area holding frequently used database pages.	Reduces disk reads.

Key Points

- Storage engines manage pages, free space and caching.
- Physical organization can strongly affect scan and range-query performance.
- Exact storage details differ by DBMS product.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Sequential scans read pages in order.
2. A buffer cache keeps hot pages in memory.
3. Heap tables are simple and flexible.
4. Ordered structures can improve range access.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 24 - PRACTICE

Database Storage and File Organization

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Rows are stored exactly in SQL result order.	Physical storage order and query result order are separate.	Use ORDER BY for guaranteed presentation order.
The buffer cache permanently stores database data.	The buffer cache is temporary memory; durable data resides on storage.	Memory contents can be lost on restart.

Guided Practice

1. What is a page?
2. What is a record?
3. What is heap organization?
4. What is a buffer cache?
5. Why does storage layout affect performance?
6. Does physical row order guarantee query order?
7. What is one benefit of caching?
8. Do all DBMS products use identical storage formats?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Database Storage and File Organization.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A fixed-size storage block used by the DBMS.
2. Stored representation of a row.
3. Storage without a logical ordering key.
4. Memory holding frequently used pages.
5. It influences I/O cost and access locality.
6. No.
7. Fewer disk reads.
8. No.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 25 - CORE DATABASE SYSTEMS

Query Processing and Optimization

Concepts, structures, rules and model examples

Learning Objectives

- Describe parsing, planning and execution.
- Explain cost-based optimization.
- Interpret simple execution plan concepts.

Concept	Explanation	Example
Parser	Checks SQL syntax and builds an internal representation.	Parses SELECT statement.
Optimizer	Chooses an execution strategy.	Selects join order and indexes.
Execution plan	Ordered set of operators used to run a query.	Scan -> Join -> Aggregate.
Cost estimate	Optimizer estimate of resources such as I/O and CPU.	Chooses lower estimated cost plan.
Statistics	Data summaries used to estimate row counts and selectivity.	Histogram or distinct-value counts.

Key Points

- A logically correct SQL query can have many possible physical execution plans.
- Accurate statistics help the optimizer estimate costs.
- Poor estimates can lead to inefficient join order or scan choices.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. The optimizer may choose a hash join for large inputs.
2. Nested-loop joins can be efficient with selective indexed lookups.
3. EXPLAIN displays a plan in many DBMSs.
4. Updating statistics may improve plan quality.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 25 - PRACTICE

Query Processing and Optimization

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
SQL text directly specifies exact physical execution steps.	SQL is mostly declarative; the optimizer chooses physical operators.	The DBMS decides how to execute the request.
The shortest SQL statement is always fastest.	Performance depends on plan, data and indexes, not text length alone.	Optimization is data-dependent.

Guided Practice

1. What does the parser do?
2. What does the optimizer do?
3. What is an execution plan?
4. What is a cost estimate?
5. Why are statistics important?
6. Name one join algorithm.
7. What does EXPLAIN show?
8. Can two equivalent queries have different performance?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Query Processing and Optimization.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Checks and interprets SQL syntax and structure.
2. Chooses an execution strategy.
3. The physical operator strategy for a query.
4. An estimate of resources used by a plan.
5. They help estimate row counts and selectivity.
6. For example, hash join or nested-loop join.
7. The query execution plan.
8. Yes.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 26 - CORE DATABASE SYSTEMS

Distributed Databases

Concepts, structures, rules and model examples

Learning Objectives

- Define distributed database concepts.
- Explain replication and partitioning.
- Identify consistency and network challenges.

Concept	Explanation	Example
Distributed database	Database whose data or processing spans multiple networked nodes.	Multi-region database cluster.
Replication	Maintaining copies of data on multiple nodes.	Primary with replicas.
Partitioning	Dividing data across nodes or storage units.	Customers partitioned by region.
Latency	Time delay in communication.	Cross-region network delay.
Consistency model	Rules defining when updates become visible across replicas.	Strong or eventual consistency.

Key Points

- Distributed systems trade off latency, availability, consistency and operational complexity.
- Replication can improve read scale and availability.
- Partitioning distributes storage and workload.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Regional replicas serve local reads.
2. A hash partition distributes rows by key.
3. Network failure can isolate nodes.
4. Conflict handling may be needed in multi-writer systems.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 26 - PRACTICE

Distributed Databases

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Replication always guarantees zero data loss.	Replication improves resilience but guarantees depend on commit and replica configuration.	Failures and lag still matter.
Partitioning means making backup copies.	Partitioning divides data; replication copies data.	They are different distribution techniques.

Guided Practice

1. What is a distributed database?
2. What is replication?
3. What is partitioning?
4. What is latency?
5. What is a consistency model?
6. Give one benefit of replication.
7. Give one benefit of partitioning.
8. Why are network failures important in distributed databases?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Distributed Databases.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A database spanning multiple networked nodes.
2. Maintaining copies of data on multiple nodes.
3. Dividing data across nodes or units.
4. Communication delay.
5. Rules for visibility and agreement of data versions.
6. Improved availability or read scale.
7. Distributed storage and workload.
8. Nodes may become unable to coordinate or receive updates.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 27 - CORE DATABASE SYSTEMS

NoSQL Databases

Concepts, structures, rules and model examples

Learning Objectives

- Explain why NoSQL systems are used.
- Distinguish document, key-value, wide-column and graph models.
- Compare schema flexibility with relational design.

Concept	Explanation	Example
Key-value store	Stores values accessed by unique keys.	SessionID -> session data.
Document database	Stores semi-structured documents, often JSON-like.	Product document with nested fields.
Wide-column database	Stores rows with flexible column families.	Large-scale event data.
Graph database	Stores nodes and relationships for highly connected data.	Social network relationships.
Schema flexibility	Ability to store records with varying structures.	Different document fields by product type.

Key Points

- NoSQL is a family of non-relational approaches, not one single model.
- Relational databases remain strong for structured data, joins and transactional integrity.
- Data model choice should follow access patterns and requirements.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A key-value store can handle session lookups.
2. A graph database can traverse relationships efficiently.
3. A document database can store nested product structures.
4. Some NoSQL systems support transactions, but guarantees differ.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 27 - PRACTICE

NoSQL Databases

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
NoSQL means no queries or no structure.	NoSQL systems have query models and structure, but not necessarily relational tables.	The term does not mean absence of data organization.
NoSQL is always better than relational databases.	Each model fits different workloads and consistency needs.	Technology choice should be requirement-driven.

Guided Practice

1. What is a key-value store?
2. What is a document database?
3. What is a graph database?
4. What is a wide-column database?
5. What is schema flexibility?
6. Give one graph database use case.
7. Does NoSQL mean no transactions?
8. How should a database model be chosen?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from NoSQL Databases.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. A system storing values by unique keys.
2. A database storing semi-structured documents.
3. A database optimized for nodes and relationships.
4. A database organized around flexible column families.
5. Ability to support varying record structures.
6. For example, social or recommendation relationships.
7. No; capabilities vary by product.
8. According to data structure, access patterns, consistency and operational requirements.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 28 - CORE DATABASE SYSTEMS

Data Warehousing and OLAP

Concepts, structures, rules and model examples

Learning Objectives

- Distinguish OLTP and OLAP workloads.
- Explain facts and dimensions.
- Describe star schema at a basic level.

Concept	Explanation	Example
OLTP	Online transaction processing for frequent operational changes.	Order entry system.
OLAP	Online analytical processing for complex analysis and reporting.	Sales trends by year and region.
Fact table	Stores measurable events and foreign keys to dimensions.	SalesFact with Amount and Quantity.
Dimension table	Stores descriptive analysis context.	Date, Product, Customer.
Star schema	Fact table connected to denormalized dimensions.	Sales star schema.

Key Points

- Warehouses integrate historical data for analysis.
- OLTP designs prioritize transaction correctness and fast updates; OLAP designs prioritize analytical reads.
- ETL or ELT processes move and transform data into analytical stores.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. A Date dimension supports analysis by month and year.
2. A Sales fact stores measures such as quantity and revenue.
3. Dashboards query warehouse aggregates.
4. Slowly changing dimensions track descriptive history.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 28 - PRACTICE

Data Warehousing and OLAP

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
A data warehouse is just a backup of the operational database.	A warehouse is organized for analysis and historical reporting.	Its purpose and schema are different.
Fact tables contain only text descriptions.	Fact tables mainly contain keys and measurable values.	Descriptive context is stored in dimensions.

Guided Practice

1. What is OLTP?
2. What is OLAP?
3. What is a fact table?
4. What is a dimension table?
5. What is a star schema?
6. Why use a data warehouse?
7. What does ETL mean?
8. Give one measure stored in a sales fact table.

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Data Warehousing and OLAP.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Operational transaction processing.
2. Analytical processing for complex queries.
3. A table of measurable events and dimension keys.
4. A table of descriptive context.
5. A fact table linked to surrounding dimensions.
6. To support integrated historical analysis.
7. Extract, Transform, Load.
8. For example, quantity or revenue.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 29 - CORE DATABASE SYSTEMS

Database APIs and Application Connectivity

Concepts, structures, rules and model examples

Learning Objectives

- Explain database drivers and connection concepts.
- Describe parameterized queries.
- Explain connection pooling and ORM at a basic level.

Concept	Explanation	Example
Driver	Software enabling an application language to communicate with a DBMS.	JDBC, ODBC or native driver.
Connection	Authenticated session between application and database.	Application opens a DB connection.
Parameterized query	SQL statement where values are supplied separately from SQL syntax.	WHERE UserID = ?.
Connection pool	Reusable set of open database connections.	Web server reuses connections.
ORM	Object-relational mapping framework linking program objects to relational tables.	Model class mapped to STUDENT.

Key Points

- Parameterized queries are a key defense against SQL injection.
- Connection pools reduce connection setup overhead.
- ORMs improve developer productivity but can generate inefficient queries if used carelessly.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Bind StudentID as a parameter rather than string concatenation.
2. Use transactions in application code for multi-step updates.
3. Close or return connections to the pool promptly.
4. Log database errors without exposing sensitive details to users.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 29 - PRACTICE

Database APIs and Application Connectivity

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Building SQL with raw user input is convenient and safe.	Use parameterized queries instead of concatenating untrusted input.	This reduces SQL injection risk.
An ORM means SQL knowledge is unnecessary.	Understanding SQL and database behavior remains important.	Generated queries still execute in the database.

Guided Practice

1. What is a database driver?
2. What is a connection?
3. What is a parameterized query?
4. What is connection pooling?
5. What is an ORM?
6. Why use parameterized queries?
7. Why use a connection pool?
8. Why should ORM users understand SQL?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Database APIs and Application Connectivity.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. Software enabling application-DBMS communication.
2. An authenticated session to the DBMS.
3. A query with values bound separately from SQL syntax.
4. Reuse of open database connections.
5. A framework mapping program objects to relational data.
6. To reduce SQL injection and improve safe value handling.
7. To reduce connection setup cost.
8. Because generated SQL affects correctness and performance.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

LESSON 30 - CORE DATABASE SYSTEMS

Database Systems Review and Exam Preparation

Concepts, structures, rules and model examples

Learning Objectives

- Review core database concepts from design to administration.
- Apply concepts to integrated scenarios.
- Use a systematic exam-answering strategy.

Concept	Explanation	Example
Design	Transforms requirements into structured schemas.	ER model -> normalized relations.
Querying	Retrieves and changes data using SQL.	SELECT, JOIN, GROUP BY.
Integrity	Protects correctness with keys and constraints.	PRIMARY KEY and FOREIGN KEY.
Transactions	Protect multi-step operations and concurrency.	ACID transaction.
Administration	Maintains security, performance, backup and availability.	DBA responsibilities.

Key Points

- Database questions often connect modeling, SQL, normalization, transactions and security.
- Use examples to justify design choices.
- For scenario questions, identify requirements before proposing technology.

Study Tip

Read each concept, connect it to a real database scenario, and create one example using a school, business or service database.

Extra Model Statements

1. Normalize a student-course design before writing joins.
2. Use a transaction for an order plus payment update.
3. Use indexes for high-value search patterns.
4. Apply least privilege and tested backups.

Database Analysis Routine

Step 1: Identify the data, rule, query or system requirement.

Step 2: Name the relevant database concept or structure.

Step 3: Explain how the concept protects correctness, performance or usability.

Step 4: Apply the rule to a concrete database example.

Your Example

Write one accurate database example and underline the key technical term.

LESSON 30 - PRACTICE

Database Systems Review and Exam Preparation

Common errors, guided practice and mastery check

Common error	Better form / idea	Why?
Memorizing SQL syntax alone is enough.	Database Systems also requires design, integrity, transactions, security and administration concepts.	The subject is broader than querying.
The fastest design is always the best design.	Correctness, maintainability, security and recoverability matter alongside performance.	System quality is multi-dimensional.

Guided Practice

1. Name five major Database Systems areas.
2. Why are keys important?
3. Why normalize data?
4. Why use transactions?
5. Why use indexes?
6. Why apply least privilege?
7. Why test backups?
8. What is the first step in a scenario design question?

Mastery Challenge

- Write two original explanations demonstrating the main ideas from Database Systems Review and Exam Preparation.
- Use correct database terminology and a realistic example.
- Check keys, relationships, constraints and SQL logic where relevant.

Quick Answers

1. For example, design, querying, integrity, transactions and administration.
2. They identify rows and enforce relationships.
3. To reduce redundancy and anomalies.
4. To protect logical units of work and concurrency.
5. To speed important access paths.
6. To limit misuse and impact of compromise.
7. To verify recovery is possible.
8. Identify requirements and business rules.

Independent Practice

- A. Write three accurate statements using the main concepts from this lesson.
- B. Design one small table, relationship or query example where appropriate.
- C. Write one deliberately incorrect statement, then correct it and explain why.
- D. Apply the lesson to a short school, business or web-application scenario of 40-60 words.

Exam Strategy

- Read command words carefully: define, describe, explain, compare, design, write or evaluate.
- Use precise database terminology and justify design choices.
- For SQL, check table names, join conditions, NULL handling and constraints.

Notes & Corrections

Record one mistake, the correction and the database rule.

REVISION 01

Review Test 1 - Foundations & Modeling

Mixed practice - complete without looking back at the lessons.

1. Introduction to Database Systems	What is a database?
2. Database Environment and Users	Who is a DBA?
3. Data Models and Database Architecture	What is a data model?
4. Entity-Relationship Modeling	Define entity.
5. Introduction to Database Systems	What is a DBMS?
6. Database Environment and Users	Give three DBA responsibilities.
7. Data Models and Database Architecture	Name three schema levels.
8. Entity-Relationship Modeling	Define attribute.
9. Introduction to Database Systems	Give two advantages of using a DBMS.
10. Database Environment and Users	What does a database designer do?
11. Data Models and Database Architecture	What does the external schema describe?
12. Entity-Relationship Modeling	Define relationship.

Applied Task

Write an 80-100 word database scenario using at least four concepts reviewed on this page. Underline the four key technical terms.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Response Space

REVISION 02

Review Test 2 - Relational Model, Keys & Algebra

Mixed practice - complete without looking back at the lessons.

1. Relational Model Fundamentals	What is a relation?
2. Keys and Integrity Constraints	What is a candidate key?
3. Relational Algebra	What does selection do?
4. Relational Model Fundamentals	What is a tuple?
5. Keys and Integrity Constraints	What is a primary key?
6. Relational Algebra	What does projection do?
7. Relational Model Fundamentals	What is an attribute?
8. Keys and Integrity Constraints	What is a foreign key?
9. Relational Algebra	What does a join do?
10. Relational Model Fundamentals	What is a domain?
11. Keys and Integrity Constraints	What is referential integrity?
12. Relational Algebra	When is union valid?

Applied Task

Write an 80-100 word database scenario using at least four concepts reviewed on this page. Underline the four key technical terms.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Response Space

REVISION 03

Review Test 3 - Core SQL

Mixed practice - complete without looking back at the lessons.

1. Introduction to SQL	What is SQL?
2. SQL SELECT and Filtering	What does SELECT do?
3. SQL Joins	What does INNER JOIN return?
4. SQL Aggregation and Grouping	What does COUNT(*) return?
5. SQL Subqueries and Set Operations	What is a subquery?
6. Introduction to SQL	What is DDL?
7. SQL SELECT and Filtering	What does WHERE do?
8. SQL Joins	What does LEFT JOIN preserve?
9. SQL Aggregation and Grouping	What does AVG do?
10. SQL Subqueries and Set Operations	What is a scalar subquery?
11. Introduction to SQL	Give two DDL statements.
12. SQL SELECT and Filtering	What does DISTINCT do?

Applied Task

Write an 80-100 word database scenario using at least four concepts reviewed on this page. Underline the four key technical terms.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Response Space

REVISION 04

Review Test 4 - Design & Normalization

Mixed practice - complete without looking back at the lessons.

1. Database Design Process	What is requirements analysis?
2. Functional Dependencies	Define functional dependency.
3. Normalization: First Normal Form	Why normalize data?
4. Normalization: Second and Third Normal Forms	Define 2NF.
5. Database Design Process	What is conceptual design?
6. Functional Dependencies	What is a determinant?
7. Normalization: First Normal Form	What is 1NF?
8. Normalization: Second and Third Normal Forms	Define 3NF.
9. Database Design Process	What is logical design?
10. Functional Dependencies	Give an example of a dependency.
11. Normalization: First Normal Form	What is a repeating group?
12. Normalization: Second and Third Normal Forms	What is a partial dependency?

Applied Task

Write an 80-100 word database scenario using at least four concepts reviewed on this page. Underline the four key technical terms.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Response Space

REVISION 05

Review Test 5 - Transactions & Performance

Mixed practice - complete without looking back at the lessons.

1. Boyce-Codd Normal Form and Denormalization	What is BCNF?
2. Transactions and ACID Properties	What is a transaction?
3. Concurrency Control	What is a lost update?
4. Indexes and Query Performance	What is an index?
5. Boyce-Codd Normal Form and Denormalization	How is BCNF related to 3NF?
6. Transactions and ACID Properties	What does atomicity mean?
7. Concurrency Control	What is a dirty read?
8. Indexes and Query Performance	What is a B-tree?
9. Boyce-Codd Normal Form and Denormalization	What is denormalization?
10. Transactions and ACID Properties	What does consistency mean?
11. Concurrency Control	What is a non-repeatable read?
12. Indexes and Query Performance	What is a full table scan?

Applied Task

Write an 80-100 word database scenario using at least four concepts reviewed on this page. Underline the four key technical terms.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Response Space

Mixed practice - complete without looking back at the lessons.

Applied Task

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

[illegible]

Mixed practice - complete without looking back at the lessons.

Applied Task	
--------------	--

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

REVISION 08

Review Test 8 - Warehousing, Applications & Project

Mixed practice - complete without looking back at the lessons.

1. NoSQL Databases	What is a key-value store?
2. Data Warehousing and OLAP	What is OLTP?
3. Database APIs and Application Connectivity	What is a database driver?
4. Database Systems Review and Exam Preparation	Name five major Database Systems areas.
5. NoSQL Databases	What is a document database?
6. Data Warehousing and OLAP	What is OLAP?
7. Database APIs and Application Connectivity	What is a connection?
8. Database Systems Review and Exam Preparation	Why are keys important?
9. NoSQL Databases	What is a graph database?
10. Data Warehousing and OLAP	What is a fact table?
11. Database APIs and Application Connectivity	What is a parameterized query?
12. Database Systems Review and Exam Preparation	Why normalize data?

Applied Task

Write an 80-100 word database scenario using at least four concepts reviewed on this page. Underline the four key technical terms.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Response Space

ANSWER KEY

Answer Key - Part 1

Use this section after completing the revision tests independently.

Review Test 1 - Foundations & Modeling

1. An organized collection of related data.	2. A person or team responsible for database administration.	3. A framework for describing data and relationships.
4. A distinguishable object or concept.	5. Software that manages database creation, storage, queries, security and maintenance.	6. Security, backup/recovery and performance monitoring, among others.
7. External, conceptual and internal.	8. A property that describes an entity or relationship.	9. For example, reduced redundancy and improved consistency.
10. Defines structures, relationships and constraints.	11. User-specific views.	12. An association between entities.

Review Test 2 - Relational Model, Keys & Algebra

1. A set of tuples sharing the same attributes.	2. A minimal set of attributes that uniquely identifies a row.	3. It chooses rows satisfying a condition.
4. A row in a relation.	5. The candidate key selected as the main identifier.	6. It chooses specified columns.
7. A named column.	8. An attribute that references a key in another relation.	9. It combines related rows from relations.
10. The allowed set or type of values for an attribute.	11. The rule that references must point to valid related rows.	12. When relations are union-compatible.

Correction Method

- For every wrong answer, write the corrected statement and the database rule that explains it.
- If an error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

Year	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	2027	2028	2029	2030	2031	2032	2033	2034	2035	2036	2037	2038	2039	2040	2041	2042	2043	2044	2045	2046	2047	2048	2049	2050	2051	2052	2053	2054	2055	2056	2057	2058	2059	2060	2061	2062	2063	2064	2065	2066	2067	2068	2069	2070	2071	2072	2073	2074	2075	2076	2077	2078	2079	2080	2081	2082	2083	2084	2085	2086	2087	2088	2089	2090	2091	2092	2093	2094	2095	2096	2097	2098	2099	2100																																																																																																													
Population	7,800,000	7,850,000	7,900,000	7,950,000	8,000,000	8,050,000	8,100,000	8,150,000	8,200,000	8,250,000	8,300,000	8,350,000	8,400,000	8,450,000	8,500,000	8,550,000	8,600,000	8,650,000	8,700,000	8,750,000	8,800,000	8,850,000	8,900,000	8,950,000	9,000,000	9,050,000	9,100,000	9,150,000	9,200,000	9,250,000	9,300,000	9,350,000	9,400,000	9,450,000	9,500,000	9,550,000	9,600,000	9,650,000	9,700,000	9,750,000	9,800,000	9,850,000	9,900,000	9,950,000	10,000,000	10,050,000	10,100,000	10,150,000	10,200,000	10,250,000	10,300,000	10,350,000	10,400,000	10,450,000	10,500,000	10,550,000	10,600,000	10,650,000	10,700,000	10,750,000	10,800,000	10,850,000	10,900,000	10,950,000	11,000,000	11,050,000	11,100,000	11,150,000	11,200,000	11,250,000	11,300,000	11,350,000	11,400,000	11,450,000	11,500,000	11,550,000	11,600,000	11,650,000	11,700,000	11,750,000	11,800,000	11,850,000	11,900,000	11,950,000	12,000,000	12,050,000	12,100,000	12,150,000	12,200,000	12,250,000	12,300,000	12,350,000	12,400,000	12,450,000	12,500,000	12,550,000	12,600,000	12,650,000	12,700,000	12,750,000	12,800,000	12,850,000	12,900,000	12,950,000	13,000,000	13,050,000	13,100,000	13,150,000	13,200,000	13,250,000	13,300,000	13,350,000	13,400,000	13,450,000	13,500,000	13,550,000	13,600,000	13,650,000	13,700,000	13,750,000	13,800,000	13,850,000	13,900,000	13,950,000	14,000,000	14,050,000	14,100,000	14,150,000	14,200,000	14,250,000	14,300,000	14,350,000	14,400,000	14,450,000	14,500,000	14,550,000	14,600,000	14,650,000	14,700,000	14,750,000	14,800,000	14,850,000	14,900,000	14,950,000	15,000,000	15,050,000	15,100,000	15,150,000	15,200,000	15,250,000	15,300,000	15,350,000	15,400,000	15,450,000	15,500,000	15,550,000	15,600,000	15,650,000	15,700,000	15,750,000	15,800,000	15,850,000	15,900,000	15,950,000	16,000,000	16,050,000	16,100,000	16,150,000	16,200,000	16,250,000	16,300,000	16,350,000	16,400,000	16,450,000	16,500,000	16,550,000	16,600,000	16,650,000	16,700,000	16,750,000	16,800,000	16,850,000	16,900,000	16,950,000	17,000,000	17,050,000	17,100,000	17,150,000	17,200,000	17,250,000	17,300,000	17,350,000	17,400,000	17,450,000	17,500,000	17,550,000	17,600,000	17,650,000	17,700,000	17,7

ANSWER KEY

Answer Key - Part 2

Use this section after completing the revision tests independently.

Review Test 3 - Core SQL

1. Structured Query Language for relational databases.	2. Returns selected columns or expressions.	3. Rows with matches on both sides.
4. Number of rows.	5. A query nested inside another statement.	6. Language for defining database structures.
7. Filters rows.	8. All rows from the left table.	9. Average of non-NULL numeric values.
10. A subquery returning one value.	11. CREATE and ALTER.	12. Removes duplicate result rows.

Review Test 4 - Design & Normalization

1. Collecting data needs, rules and tasks.	2. A relationship where one attribute set determines another.	3. To reduce redundancy and update anomalies.
4. 1NF with no partial dependency of non-key attributes on part of a candidate key.	5. Creating a high-level model such as an ER model.	6. The attribute set on the left side.
7. Atomic values with no repeating groups.	8. 2NF with no inappropriate transitive dependency of non-key attributes on keys.	9. Mapping the conceptual model to database structures.
10. For example, StudentID -> StudentName.	11. Repeated similar attributes or multiple values in one row structure.	12. A dependency on part of a composite key.

Correction Method

- For every wrong answer, write the corrected statement and the database rule that explains it.
- If an error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

Year	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	2027	2028	2029	2030	2031	2032	2033	2034	2035	2036	2037	2038	2039	2040	2041	2042	2043	2044	2045	2046	2047	2048	2049	2050	2051	2052	2053	2054	2055	2056	2057	2058	2059	2060	2061	2062	2063	2064	2065	2066	2067	2068	2069	2070	2071	2072	2073	2074	2075	2076	2077	2078	2079	2080	2081	2082	2083	2084	2085	2086	2087	2088	2089	2090	2091	2092	2093	2094	2095	2096	2097	2098	2099	2100																																																																																																																																																																																																																																		
Population	7.5	7.6	7.7	7.8	7.9	8.0	8.1	8.2	8.3	8.4	8.5	8.6	8.7	8.8	8.9	9.0	9.1	9.2	9.3	9.4	9.5	9.6	9.7	9.8	9.9	10.0	10.1	10.2	10.3	10.4	10.5	10.6	10.7	10.8	10.9	11.0	11.1	11.2	11.3	11.4	11.5	11.6	11.7	11.8	11.9	12.0	12.1	12.2	12.3	12.4	12.5	12.6	12.7	12.8	12.9	13.0	13.1	13.2	13.3	13.4	13.5	13.6	13.7	13.8	13.9	14.0	14.1	14.2	14.3	14.4	14.5	14.6	14.7	14.8	14.9	15.0	15.1	15.2	15.3	15.4	15.5	15.6	15.7	15.8	15.9	16.0	16.1	16.2	16.3	16.4	16.5	16.6	16.7	16.8	16.9	17.0	17.1	17.2	17.3	17.4	17.5	17.6	17.7	17.8	17.9	18.0	18.1	18.2	18.3	18.4	18.5	18.6	18.7	18.8	18.9	19.0	19.1	19.2	19.3	19.4	19.5	19.6	19.7	19.8	19.9	20.0	20.1	20.2	20.3	20.4	20.5	20.6	20.7	20.8	20.9	21.0	21.1	21.2	21.3	21.4	21.5	21.6	21.7	21.8	21.9	22.0	22.1	22.2	22.3	22.4	22.5	22.6	22.7	22.8	22.9	23.0	23.1	23.2	23.3	23.4	23.5	23.6	23.7	23.8	23.9	24.0	24.1	24.2	24.3	24.4	24.5	24.6	24.7	24.8	24.9	25.0	25.1	25.2	25.3	25.4	25.5	25.6	25.7	25.8	25.9	26.0	26.1	26.2	26.3	26.4	26.5	26.6	26.7	26.8	26.9	27.0	27.1	27.2	27.3	27.4	27.5	27.6	27.7	27.8	27.9	28.0	28.1	28.2	28.3	28.4	28.5	28.6	28.7	28.8	28.9	29.0	29.1	29.2	29.3	29.4	29.5	29.6	29.7	29.8	29.9	30.0	30.1	30.2	30.3	30.4	30.5	30.6	30.7	30.8	30.9	31.0	31.1	31.2	31.3	31.4	31.5	31.6	31.7	31.8	31.9	32.0	32.1	32.2	32.3	32.4	32.5	32.6	32.7	32.8	32.9	33.0	33.1	33.2	33.3	33.4	33.5	33.6	33.7	33.8	33.9	34.0	34.1	34.2	34.3	34.4	34.5	34.6	34.7	34.8	34.9	35.0	35.1	35.2	35.3	35.4	35.5	35.6	35.7	35.8	35.9	36.0	36.1	36.2	36.3	36.4	36.5	36.6	36.7	36.8	36.9	37.0	37.1	37.2	37.3	37.4	37.5	37.6	37.7	37.8	37.9	38.0	38.1	38.2	38.3	38.4	38.5	38.6	38.7	38.8	38.9	39.0	39.1

ANSWER KEY

Answer Key - Part 3

Use this section after completing the revision tests independently.

Review Test 5 - Transactions & Performance

1. Every determinant of a non-trivial dependency must be a superkey.	2. A logical unit of database work.	3. An update is overwritten by another concurrent update.
4. A structure that speeds selected data access.	5. BCNF is stricter in some dependency cases.	6. All operations succeed together or none are applied.
7. Reading uncommitted data.	8. A balanced tree index structure.	9. Intentional introduction of redundancy.
10. Defined rules remain valid.	11. Reading the same row twice and seeing different committed values.	12. Reading many or all rows directly.

Review Test 6 - Security, Recovery & Storage

1. A stored query presented like a virtual table.	2. Verification of identity.	3. A copy of data for restoration.
4. A named database program for operations.	5. Determination of allowed actions.	6. Restoring data and database service after failure.
7. A routine that returns a value or result.	8. A named collection of privileges.	9. A backup of all selected data.
10. An automatic action fired by a database event.	11. Permission to perform a database action.	12. A backup of changes according to the backup method.

Correction Method

- For every wrong answer, write the corrected statement and the database rule that explains it.
- If an error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

ANSWER KEY

Answer Key - Part 4

Use this section after completing the revision tests independently.

Review Test 7 - Optimization, Distributed & NoSQL

1. A fixed-size storage block used by the DBMS.	2. Checks and interprets SQL syntax and structure.	3. A database spanning multiple networked nodes.
4. Stored representation of a row.	5. Chooses an execution strategy.	6. Maintaining copies of data on multiple nodes.
7. Storage without a logical ordering key.	8. The physical operator strategy for a query.	9. Dividing data across nodes or units.
10. Memory holding frequently used pages.	11. An estimate of resources used by a plan.	12. Communication delay.

Review Test 8 - Warehousing, Applications & Project

1. A system storing values by unique keys.	2. Operational transaction processing.	3. Software enabling application-DBMS communication.
4. For example, design, querying, integrity, transactions and administration.	5. A database storing semi-structured documents.	6. Analytical processing for complex queries.
7. An authenticated session to the DBMS.	8. They identify rows and enforce relationships.	9. A database optimized for nodes and relationships.
10. A table of measurable events and dimension keys.	11. A query with values bound separately from SQL syntax.	12. To reduce redundancy and anomalies.

Correction Method

- For every wrong answer, write the corrected statement and the database rule that explains it.
- If an error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

ANSWER KEY

Answer Key - Part 5: SQL & Modeling Skills

A compact reference for practical database work.

Core SQL Patterns

Concept	Explanation	Example
Create table	CREATE TABLE Student (StudentID INT PRIMARY KEY, Name VARCHAR(100));	Defines a relation and constraint.
Insert	INSERT INTO Student (StudentID, Name) VALUES (1, 'Ana');	Adds one row.
Select	SELECT Name FROM Student WHERE StudentID = 1;	Retrieves filtered data.
Join	SELECT s.Name, e.CourseID FROM Student s JOIN Enrollment e ON e.StudentID = s.StudentID;	Combines related tables.
Group	SELECT Class, COUNT(*) FROM Student GROUP BY Class;	Aggregates by group.

Modeling Checklist

- Identify entities and keys.
- Model one-to-many relationships with foreign keys.
- Resolve many-to-many relationships with an associative table.
- Remove repeating groups and partial/transitive redundancy.
- Document domains, constraints and business rules.

SQL Safety Checklist

- Use parameterized queries for application input.
- Avoid SELECT * in stable application interfaces when only specific columns are needed.
- Check NULL handling explicitly.
- Use transactions for multi-step operations.
- Test UPDATE and DELETE predicates carefully.

Practice Prompts

1. Write a table design for COURSE with a primary key.
2. Write a query that lists students in one class.
3. Explain why ENROLLMENT needs foreign keys.
4. Explain why a multi-step payment update should use a transaction.

ANSWER KEY

Answer Key - Part 6: Command Words & Key Definitions

Use precise language to match the demand of the question.

Common Command Words

Concept	Explanation	Example
Define	Give the precise meaning of a database term.	
Describe	Give features or state what happens.	
Explain	Give reasons and connect cause to effect.	
Compare	Give similarities and differences.	
Design	Create a justified structure that meets requirements.	
Write SQL	Provide syntactically correct SQL for the task.	
Evaluate	Discuss strengths, limitations and trade-offs.	
Justify	Give reasons supporting a design or technology choice.	

Key Definitions

Primary key	Chosen candidate key that uniquely identifies each row.
Foreign key	Attribute that references a key in another relation.
Normalization	Structuring relations to reduce redundancy and anomalies.
Transaction	Logical unit of database work.
Index	Auxiliary structure that speeds selected data access.
View	Stored query presented like a virtual table.
Replication	Maintaining copies of data on multiple nodes.
Least privilege	Granting only the access needed for a task.

Final Revision Routine

- Redraw key ER models from memory.
- Normalize sample tables to 3NF.
- Practice SELECT, JOIN, GROUP BY and subqueries.
- Explain ACID, indexes, security and backup in your own words.

REFERENCE

Database Systems Reference Tables

Core SQL, normalization and transaction reminders.

Essential SQL Clauses

Concept	Explanation	Example
SELECT	Choose result columns or expressions.	SELECT Name FROM Student;
FROM	Identify source tables or views.	FROM Student s
JOIN ... ON	Combine related rows.	JOIN Enrollment e ON e.StudentID = s.StudentID
WHERE	Filter individual rows.	WHERE Class = 'Terminale'
GROUP BY	Form groups for aggregates.	GROUP BY Class
HAVING	Filter groups after aggregation.	HAVING COUNT(*) > 10
ORDER BY	Sort the final result.	ORDER BY Name ASC

Normalization Summary

Form	Main requirement
1NF	Atomic values; no repeating groups.
2NF	1NF plus no partial dependency of non-key attributes on part of a candidate key.
3NF	2NF plus no inappropriate transitive dependency of non-key attributes.
BCNF	Every determinant of a non-trivial dependency is a superkey.

ACID Properties

- Atomicity: all or nothing.
- Consistency: rules remain valid.
- Isolation: concurrent transactions do not improperly interfere.
- Durability: committed data survives failures.

Key Relationship Reminder

Primary keys identify rows. Foreign keys connect related rows. Composite keys use more than one attribute. Many-to-many relationships are normally represented by an associative table containing foreign keys to both parent tables.

REFERENCE

Database Systems Quick Guide

A final one-page checklist before an examination.

Design

- Start with requirements and business rules.
- Model entities, attributes, relationships and cardinality.
- Choose primary and foreign keys.
- Normalize to reduce redundancy and anomalies.

SQL

- SELECT retrieves data.
- WHERE filters rows.
- JOIN combines related tables.
- GROUP BY and HAVING support aggregation.

Transactions

- Use transactions for multi-step logical work.
- COMMIT saves changes; ROLLBACK cancels uncommitted work.
- Know ACID properties.
- Understand dirty reads, lost updates and deadlocks.

Performance

- Indexes speed suitable reads but cost storage and write maintenance.
- Optimizers choose query plans using statistics.
- Do not assume row order without ORDER BY.
- Measure before denormalizing.

Security & Recovery

- Authenticate identities and authorize actions.
- Apply least privilege and roles.
- Use tested backups and recovery procedures.
- Replication improves availability but does not replace backups.

Modern Systems

- Distributed databases use replication and partitioning.
- NoSQL includes key-value, document, wide-column and graph models.
- Data warehouses support analytical workloads.
- Application code should use parameterized queries.

Final Exam Checklist

- Answer exactly what the command word asks.
- Use correct database terminology.
- For design questions, justify keys and relationships.
- For SQL, check joins, filters, grouping and NULL logic.
- For system questions, discuss trade-offs in correctness, security and performance.